

EPIC EVENTS

Architecture technique et sécurité

CRM en ligne de commande - Python, Peewee, SQLite, Typer, Sentry et Pytest

Projet OpenClassrooms P12
Juillet 2026

Résultats validés : 147 tests réussis, 0 échec, 76 % de couverture globale et réception effective d'un événement de démonstration dans Sentry.

Sommaire

1. Synthèse de validation
2. Vue d'ensemble de l'architecture
3. Schéma de la base de données
4. Authentification et permissions
5. Sécurité et supervision avec Sentry
6. Stratégie de tests et couverture
7. Installation et démonstration
8. Vérification finale du dépôt

1. Synthèse de validation

Le projet Epic Events met en œuvre une architecture en couches, une base relationnelle SQLite, l'authentification et l'autorisation par rôle, le principe du moindre privilège, la journalisation Sentry et une suite de tests automatisés.

- 147 tests réussis ;
- 0 échec ;
- 76 % de couverture globale ;
- événement de démonstration reçu dans Sentry ;
- base de tests SQLite isolée en mémoire ;
- secrets et base locale exclus du dépôt Git.

2. Vue d'ensemble de l'architecture

2.1 Objectif

Epic Events est un CRM sécurisé en ligne de commande utilisé par trois départements : gestion, commercial et support. L'architecture sépare l'interaction terminal, l'authentification, l'autorisation, les règles métier, la persistance et la supervision.

2.2 Composants principaux

```

Couche CLI
src/main.py
|
+--> authentification et autorisation
|   src/auth.py
|
+--> services métier
    src/services/
        |
        v
    modèles Peewee
    src/models/
        |
        v
    base SQLite
    epic_events.db

Exceptions inattendues
|
v
src/monitoring.py
|
v
Sentry
  
```

2.3 Responsabilités des couches

Couche	Responsabilités
CLI	Définit les commandes Typer, collecte les arguments, demande les mots de passe de manière masquée, convertit les entrées et affiche les résultats.
Authentification	Identifie l'utilisateur, vérifie le hash du mot de passe, contrôle le statut actif et les rôles.

Services métier	Valide les données, applique les règles de rôle, de propriété et d'affectation, puis exécute les opérations métier.
Persistance	Définit les modèles Peewee, les relations et la connexion SQLite.
Supervision	Initialise Sentry, capture les exceptions inattendues et vide la file d'envoi avant l'arrêt du processus.

2.4 Flux d'une requête

1. Le collaborateur lance une commande avec son email.
2. La CLI demande le mot de passe dans un prompt masqué.
3. src/auth.py vérifie le hash bcrypt et le statut actif.
4. La CLI transmet l'utilisateur authentifié au service métier.
5. Le service contrôle le rôle, la propriété et les données.
6. Peewee exécute une requête paramétrée.
7. La CLI affiche le résultat ou une erreur contrôlée.

2.5 Structure du code

```
src/
├── auth.py
├── config.py
├── database.py
├── main.py
├── monitoring.py
├── models/
│   ├── user.py
│   ├── client.py
│   ├── contract.py
│   └── event.py
├── services/
│   ├── user_service.py
│   ├── client_service.py
│   ├── contract_service.py
│   └── event_service.py
```

2.6 Propriétés architecturales

- séparation explicite des responsabilités ;
- autorisations appliquées dans la couche service ;
- absence de SQL construit directement avec les entrées utilisateur ;
- absence de mots de passe en clair ;
- absence de secret codé en dur ;
- base de tests isolée ;
- capture centralisée des exceptions inattendues ;
- décisions techniques traçables dans les ADR.

3. Schéma de la base de données

3.1 Moteur et ORM

L'application utilise SQLite avec l'ORM Peewee. SQLite est adapté à une application locale en ligne de commande, ne nécessite pas de serveur de base de données et facilite l'installation dans un environnement vierge.

- définition du schéma à partir des modèles ;

- requêtes paramétrées ;
- relations par clés étrangères ;
- opérations lisibles dans la couche service ;
- intégration simple avec SQLite.

3.2 Diagramme des relations

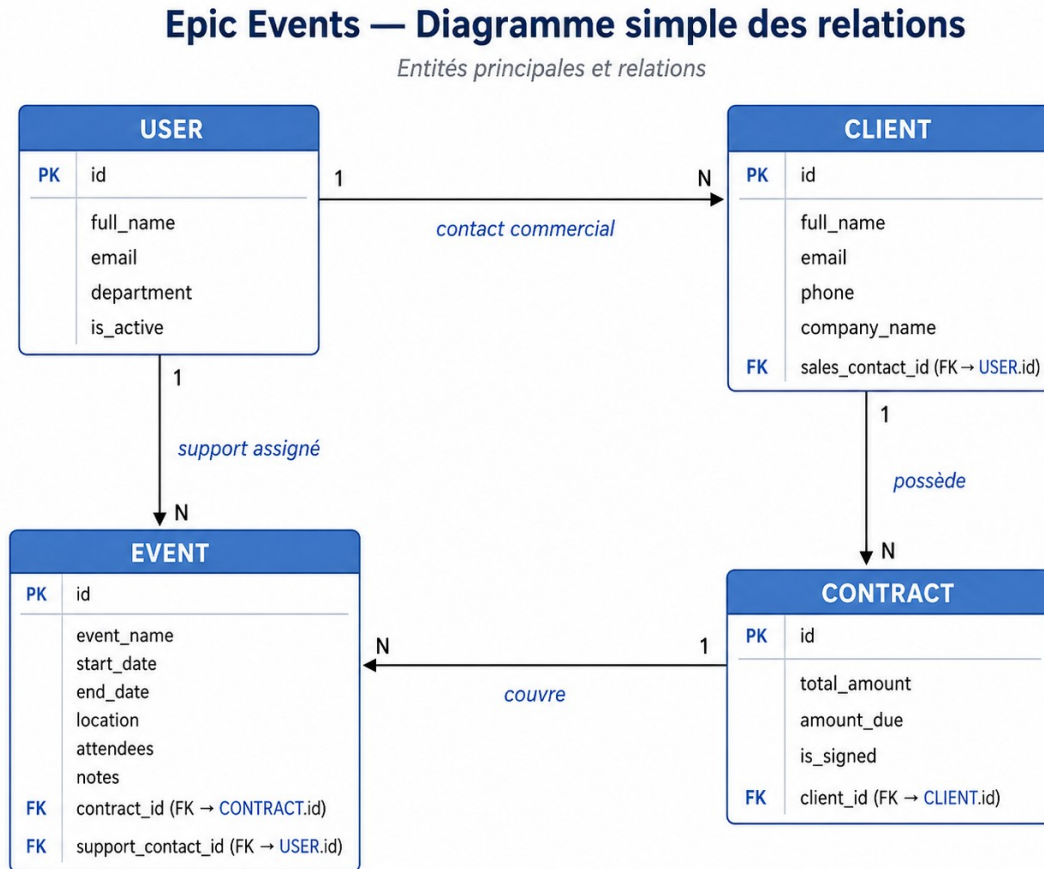


Figure 1 - Relations principales entre User, Client, Contract et Event

3.3 Tables

3.3.1 users

Champ	Type	Contrainte	Rôle
id	Integer	Clé primaire	Identifiant du collaborateur
full_name	Varchar(255)	Obligatoire	Nom affiché
email	Varchar(255)	Unique, obligatoire	Identifiant de connexion
password_hash	Varchar(255)	Obligatoire	Hash bcrypt
department	Varchar(50)	Obligatoire	MANAGEMENT, COMMERCIAL ou SUPPORT
is_active	Boolean	Vrai par défaut	Statut logique du compte

3.3.2 clients

Champ	Type	Contrainte	Rôle
id	Integer	Clé primaire	Identifiant du client
full_name	Varchar(255)	Obligatoire	Nom du client
email	Varchar(255)	Unique, obligatoire	Email du client

phone	Varchar(30)	Obligatoire	Téléphone
company_name	Varchar(255)	Obligatoire	Entreprise
created_at	Datetime	Valeur automatique	Premier contact
updated_at	Datetime	Mis à jour par service	Dernière modification
sales_contact_id	Integer	FK users, RESTRICT	Commercial responsable

3.3.3 contracts

Champ	Type	Contrainte	Rôle
id	Integer	Clé primaire	Identifiant du contrat
client_id	Integer	FK clients, RESTRICT	Client concerné
sales_contact_id	Integer	FK users, RESTRICT	Contact commercial
total_amount	Decimal(10,2)	Strictement positif	Montant total
amount_due	Decimal(10,2)	Entre 0 et total	Reste à payer
is_signed	Boolean	Faux par défaut	Statut de signature
created_at	Datetime	Valeur automatique	Date de création

3.3.4 events

Champ	Type	Contrainte	Rôle
id	Integer	Clé primaire	Identifiant de l'événement
contract_id	Integer	FK contracts, RESTRICT	Contrat signé
support_contact_id	Integer	FK users nullable, SET NULL	Support affecté
event_name	Varchar(255)	Obligatoire	Nom de l'événement
location	Varchar(255)	Obligatoire	Lieu
attendees	Integer	Supérieur à zéro	Nombre de participants
event_start	Datetime	Obligatoire	Début
event_end	Datetime	Après le début	Fin
notes	Text	Facultatif	Notes opérationnelles

3.4 Logique des relations

- un commercial peut être responsable de plusieurs clients ;
- un client peut posséder plusieurs contrats ;
- le contrat conserve le commercial associé au client ;
- un contrat peut être lié à plusieurs événements ;
- un événement peut temporairement ne pas avoir de support ;
- un support peut être affecté à plusieurs événements ;
- la désactivation d'un compte préserve les données liées.

3.5 Intégrité et protection contre les injections

- validation des entrées avant persistance ;
- unicité des emails ;
- contrôles de cohérence sur les montants et les dates ;
- clés étrangères pour préserver les relations ;
- valeurs compilées comme paramètres de requête par Peewee.

4. Authentification et permissions

4.1 Authentification

Chaque commande protégée reçoit l'email du collaborateur puis demande son mot de passe via un prompt masqué.

```

email + mot de passe masqué
|
v
authenticate_user()
|
+--> email inconnu : refus
+--> mot de passe invalide : refus
+--> compte inactif : refus
+--> compte valide et actif : utilisateur retourné

```

- les mots de passe ne sont jamais stockés en clair ;
- ils ne sont jamais passés dans les arguments de commande ;
- ils sont hachés avec Passlib et bcrypt ;
- le hash enregistré est vérifié à l'authentification.

4.2 Autorisation

L'authentification répond à la question « Qui est le collaborateur ? ». L'autorisation répond à la question « A-t-il le droit d'effectuer cette opération ? ». Les contrôles génériques sont centralisés dans `src/auth.py` et les règles métier sont appliquées par les services.

4.3 Matrice des permissions

Opération	Gestion	Commercial	Support
Lire tous les clients	Oui	Oui	Oui
Lire tous les contrats	Oui	Oui	Oui
Lire tous les événements	Oui	Oui	Oui
Créer, modifier ou désactiver un collaborateur	Oui	Non	Non
Créer un client	Non	Oui	Non
Modifier un client	Non	Ses clients	Non
Créer un contrat	Oui	Non	Non
Modifier un contrat	Tous	Clients propres	Non
Lister les contrats non signés ou non payés	Non	Oui	Non
Créer un événement	Non	Client propre + contrat signé	Non
Lister les événements sans support	Oui	Non	Non
Affecter un support	Oui	Non	Non
Lister ou modifier ses événements	Non	Non	Affectations propres
Déclencher le test Sentry contrôlé	Oui	Non	Non

4.4 Principe du moindre privilège

- la gestion administre les collaborateurs, les contrats et les affectations support, sans créer les clients ni les événements commerciaux ;
- le commercial agit seulement sur ses clients et les contrats associés ;
- le commercial crée un événement uniquement pour un contrat signé de son client ;
- le support consulte et modifie uniquement les événements qui lui sont affectés.

4.5 Désactivation des comptes

La suppression d'un collaborateur est une désactivation logique. Les clients, contrats et événements liés sont conservés. Un compte désactivé ne peut plus s'authentifier ni exercer de permission.

5. Sécurité et supervision avec Sentry

5.1 Contrôles de sécurité

Risque	Mesure
Mot de passe exposé	Prompt masqué, hash bcrypt et aucune valeur sensible dans l'historique de commande.
Injection SQL	Requêtes Peewee paramétrées au lieu de chaînes SQL assemblées.
Élévation de privilège	Contrôles de rôle dans auth.py et les services.
Accès à une donnée tierce	Contrôles de propriété et d'affectation.
Secret committé	Exclusion de .env, du DSN réel, de la base locale et des rapports générés.
Fuite par monitoring	send_default_pii=False et include_local_variables=False.

5.2 Validation des données

- champs obligatoires ;
- emails uniques ;
- départements autorisés ;
- montants de contrat cohérents ;
- statut de signature booléen ;
- règles de propriété ;
- nombre de participants positif ;
- date de fin postérieure à la date de début ;
- statut actif du collaborateur.

5.3 Architecture Sentry

```

SENTRY_DSN
SENTRY_ENVIRONMENT
|
v
src/config.py
|
v
initialize_sentry()
|
+--> aucun DSN : supervision désactivée sans erreur
+--> DSN configuré : sentry_sdk.init()
```


5.4 Gestion des exceptions

```
run_app()
|
v
application Typer
|
+--> erreur attendue de validation ou de permission
|   message explicite dans la CLI
|
+--> exception technique inattendue
    capture_application_exception()
    |
    v
    Sentry
```

5.5 Démonstration contrôlée

```
python -m src.main test-sentry-command manager@epicevents.com
```

Cette commande réservée à la gestion génère une `RuntimeError` contrôlée :

```
Epic Events controlled Sentry demonstration error.
```

La validation réelle a confirmé l'initialisation du SDK, le retour d'un identifiant d'événement et l'apparition du problème dans le tableau de bord Sentry.

5.6 Erreurs attendues et inattendues

Les identifiants invalides, champs manquants, opérations non autorisées, montants incorrects ou dates incohérentes sont des erreurs métier contrôlées. Les exceptions techniques inattendues sont capturées au niveau de l'entrée principale et transmises à Sentry.

6. Stratégie de tests et couverture

6.1 Objectifs

- authentification ;
- autorisation ;
- restrictions par rôle et propriété ;
- persistance avec Peewee ;
- validation des entrées ;
- parcours de la CLI ;
- comportement de l'intégration Sentry ;
- initialisation de la base ;
- absence de régression.

6.2 Organisation

```
tests/
├── conftest.py
├── unit/
│   └── test_monitoring.py
├── integration/
│   ├── test_auth.py
│   ├── test_client_service.py
│   ├── test_contract_service.py
│   ├── test_database.py
│   ├── test_event_service.py
│   └── test_user_service.py
└── functional/
    └── test_cli.py
```

6.3 Résultats par catégorie

Catégorie	Périmètre	Résultat
Unitaires	Monitoring isolé avec des mocks	4 réussis
Intégration	Authentification, services, modèles et SQLite en mémoire	134 réussis
Fonctionnels	CLI Typer exécutée avec CliRunner	9 réussis
Total	Suite complète	147 réussis, 0 échec

6.4 Isolation de la base

Les tests automatisés utilisent `SqliteDatabase(":memory:")` et lient temporairement les modèles à cette base. Le fichier `epic_events.db` n'est pas ouvert, chaque test reçoit un schéma propre et les données de démonstration locales sont protégées.

6.5 Commandes

```
python -m pytest -v

python -m pytest tests/unit -v
python -m pytest tests/integration -v
python -m pytest tests/functional -v

python -m pytest --cov=src --cov-report=term-missing --cov-report=html
```

6.6 Couverture validée

Composant	Couverture
Modèles	100 %
Service clients	100 %
Service contrats	100 %
Service utilisateurs	100 %
Authentification	97 %
Service événements	97 %
Monitoring	95 %
CLI principale	41 %
TOTAL	76 %

Le fichier de CLI contient de nombreuses branches d'affichage et de gestion d'erreurs. Les couches métier et de sécurité conservent une couverture élevée.

7. Installation et démonstration

7.1 Prérequis

- Git ;
- Python 3.12 recommandé ;
- connexion internet pour installer les dépendances ;
- projet Sentry facultatif pour la démonstration.

7.2 Installation dans un environnement vierge

```
git clone https://github.com/AlNocquet/OC-Projet12-EpicEvents.git
cd OC-Projet12-EpicEvents

python -m venv venv
source venv/Scripts/activate

python -m pip install --upgrade pip
python -m pip install -r requirements.txt

python -m src.main initialize-database

python -m src.main create-initial-management-user-command \
"Morgan Manager" \
manager@epicevents.com
```

Sous PowerShell, utiliser `.\venv\Scripts\Activate.ps1`. Sous macOS ou Linux, utiliser `source venv/bin/activate`. Le mot de passe du premier compte est saisi et confirmé dans des prompts masqués.

7.3 Vérification de l'installation

- environnement virtuel créé ;
- dépendances installées sans erreur ;
- tables SQLite créées ;
- premier compte de gestion créé ;
- authentification réussie ;
- commandes CRM visibles avec `python -m src.main --help` ;
- suite de tests complète réussie ;
- rapport de couverture généré ;
- aucun secret committé.

7.4 Proposition de démonstration en dix minutes

Étape	Démonstration	Point expliqué
1	Architecture et schéma	Relations, clés étrangères, choix SQLite et Peewee.
2	Authentification	Prompt masqué, hash bcrypt, compte actif, différence avec l'autorisation.
3	Parcours gestion	Création d'un collaborateur et d'un contrat, affectation support.
4	Parcours commercial	Création d'un client, filtres de contrats, création d'un événement signé.
5	Parcours support	Liste et mise à jour des événements affectés.
6	Sécurité	ORM, moindre privilège, validation, exclusions Git.
7	Tests et couverture	147 tests réussis et 76 % de couverture.
8	Sentry	Exception contrôlée visible dans le tableau de bord.

8. Vérification finale du dépôt

```
git status
git log --oneline --decorate --graph --all
```

8.1 Résultat attendu

- arbre de travail propre ;
- branches fonctionnelles cohérentes ;
- absence de fichier .env ;
- absence de base locale ;
- absence d'artefacts de couverture ;
- README complet ;
- schéma de base de données et documentation d'architecture présents.

8.2 Conclusion

L'architecture Epic Events répond aux exigences fonctionnelles, techniques et de sécurité du CRM. Le projet fournit une base relationnelle documentée, une authentification robuste, des permissions de moindre privilège, une supervision Sentry, une suite de tests organisée et une procédure d'installation reproductible.

Validation finale : 147 tests réussis, 0 échec, 76 % de couverture et démonstration Sentry confirmée.